

Module 12 — Greedy Algorithms

Sources: CLRS 4e ch. 15 (Greedy Algorithms) · Skiena 3e §1.2 (Selecting the Right Jobs), §21.5 (Text Compression), §8.1 (MST as greedy)

Big Idea

"A greedy algorithm always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice leads to a globally optimal solution."

That's the definition. The entire subject is the word "**hope**" — and the discipline of replacing hope with proof.

Greedy and dynamic programming both rest on **optimal substructure**. The difference is *when the choice is made*:

	DYNAMIC PROGRAMMING	GREEDY
Order of operations	solve subproblems, then choose	choose, then solve one subproblem
Direction	bottom-up (or top-down memoized)	top-down
The choice depends on	the solutions to subproblems	only the input and choices already made — never on future choices or subproblem solutions
Subproblems after a choice	possibly many	exactly one
Cost	usually polynomial with a table	usually a sort or a priority queue

CLRS's summary sentence: *"a dynamic-programming algorithm proceeds bottom up, whereas a greedy strategy usually progresses top down, making one greedy choice after another, reducing each given problem instance to a smaller one."* And the crucial caveat: "**beneath every greedy algorithm, there is almost always a more cumbersome dynamic-programming solution.**"

Greedy is the reward for a proof. Skiena's §1.2 is the best cautionary tale in either book: he proposes two perfectly reasonable-sounding greedy heuristics for interval scheduling, and **kills both with pictures**. The third one is correct — but only because he argues it, not because it sounds good. **A greedy algorithm without a proof is a heuristic, and you should call it that.**

Remember months later: *greedy = an exchange argument. Take any optimal solution, show you can swap your greedy choice in without making it worse, and induct. If you can't produce that argument in two sentences, your greedy algorithm is probably wrong — go find the counterexample or fall back to DP.*

What You Should Be Able To Do After This Chapter

- Derive a greedy algorithm from a DP formulation by observing that only one choice needs to be considered.
 - State the **greedy-choice property** and prove it with an **exchange argument** (the "cut-and-paste for greedy").
 - Reproduce Theorem 15.1 (earliest-finish-first is safe) and the Huffman lemmas from memory.
 - Produce counterexamples for the *wrong* greedy heuristics: earliest start, shortest job, fewest conflicts, highest value density.
 - Explain precisely why greedy solves fractional knapsack but not 0-1 knapsack.
 - Build a Huffman code, prove it optimal, and state its three practical disadvantages.
 - Prove that furthest-in-future is the optimal offline cache eviction policy, and explain why we study an offline problem at all.
 - Recognize the standard greedy families: interval scheduling, exchange-argument scheduling, matroid-shaped problems (MST, task scheduling), and the ones that only *look* greedy.
 - Decide, in an interview, whether to attempt greedy or go straight to DP.
-

Part 1 — Activity Selection: Watching Greedy Emerge From DP

The problem

You are scheduling one conference room. $S = \{a_1, \dots, a_n\}$, activity a_i occupies the half-open interval $[s_i, f_i)$. Two activities are **compatible** if their intervals don't overlap ($s_i \geq f_j$ or $s_j \geq f_i$). **Select a maximum-size set of mutually compatible activities.**

Assume the input is sorted by finish time: $f_1 \leq f_2 \leq \dots \leq f_n$. (If it isn't, sort in $O(n \lg n)$. "We'll see later the advantage that this assumption provides" — and it's a big one.)

CLRS's running instance:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

$\{a_3, a_9, a_{11}\}$ is compatible but not maximum. $\{a_1, a_4, a_8, a_{11}\}$ is — and so is $\{a_2, a_4, a_9, a_{11}\}$. **Note there are multiple optima; greedy finds one of them.**

Step 1: the DP formulation (which we will then throw away)

Let S_{ij} = activities that start after a_i finishes and finish before a_j starts. If a maximum set $A_{ij} \subseteq S_{ij}$ contains a_k , then $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$, so $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$. The **cut-and-paste** argument (M11) shows A_{ik} and A_{kj} must themselves be optimal. Hence:

$$c[i,j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset \\ \max\{c[i,k] + c[k,j] + 1 : a_k \in S_{ij}\} & \text{if } S_{ij} \neq \emptyset \end{cases} \quad (15.2)$$

That's a perfectly good $O(n^3)$ DP. **But it overlooks something.**

Step 2: the greedy choice

"What if you could choose an activity to add to an optimal solution without having to first solve all the subproblems?"

Intuition: choose the activity that leaves the resource available for as many other activities as possible. Of whatever activities you end up choosing, one must finish first — so pick **the activity with the earliest finish time**. Since the input is sorted by finish time, **the greedy choice is always a_1** .

And then only one subproblem remains. Why don't you need to consider activities finishing *before* a_1 starts? Because $s_1 < f_1$ and f_1 is the earliest finish time of any activity — so **no activity can finish at or before s_1** . Everything compatible with a_1 starts after a_1 finishes. The two-index subproblem space $S_{i,j}$ collapses to the one-index space $S_k = \{a_i \in S : s_i \geq f_k\}$.

Theorem 15.1. Consider any nonempty subproblem S_k , and let a_m be an activity in S_k with the **earliest finish time**. Then a_m is included in **some** maximum-size subset of mutually compatible activities of S_k .

Proof (the exchange argument — this is the template). Let A_k be any maximum-size compatible subset of S_k , and let a_j be the activity in A_k with the earliest finish time.

- If $a_j = a_m$, done.
- Otherwise, form $A'_k = (A_k - \{a_j\}) \cup \{a_m\}$. The activities in A'_k are still compatible: the rest of A_k was compatible with a_j , a_j was the *first* to finish in A_k , and $f_m \leq f_j$ — so a_m finishes no later than a_j and therefore clashes with nothing that a_j didn't clash with. Since $|A'_k| = |A_k|$, A'_k is also maximum, and it contains a_m . ■

Read the shape of that proof, because you will write it a dozen times: take an arbitrary optimum, swap in the greedy choice, show the result is still feasible and no worse.

Note the exact wording: "**is included in some maximum-size subset**" — not "in every optimal solution". That distinction is what makes the argument work at all.

Steps 3–4: the algorithms

<pre> RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n) SELECTOR(s, f, n) 1 m = k + 1 2 while m ≤ n and s[m] < f[k] 3 m = m + 1 // first in S_k 4 if m ≤ n // is a_m in S_k? 5 return {a_m} U RECURSIVE-...(s, f, m, n) {a_m} 6 else return ∅ </pre>	<pre> GREEDY-ACTIVITY- 1 A = {a₁} 2 k = 1 3 for m = 2 to n 4 if s[m] ≥ f[k] 5 A = A U 6 k = m 7 return A </pre>
--	---

→ **C++ implementation:** [A1 RECURSIVE-ACTIVITY-SELECTOR](#) and [GREEDY-ACTIVITY-SELECTOR](#)

Start with a fictitious a_0 with $f_0 = 0$, so S_0 is everything; the initial call is `RECURSIVE-ACTIVITY-SELECTOR(s, f, 0, n)`.

Both run in $\Theta(n)$ after sorting. For the recursive version: *"over all recursive calls, each activity is examined exactly once in the while loop test of line 2."* The recursive version is *almost* tail-recursive (a recursive call followed by a union), which is why the conversion to a loop is mechanical.

The key invariant of the iterative version: k indexes the most recent addition to A , and because activities are processed in finish-time order,

$$f_k = \max\{ f_i : a_i \in A \} \quad (15.3)$$

so checking $s[m] \geq f[k]$ against **just the last chosen activity** suffices to establish compatibility with **all** of them. That is what turns an $O(n^2)$ compatibility check into $O(1)$.

Skiena's version: the movie scheduling problem

Same problem, better story. You are a highly in-demand actor with n movie offers, each with a first and last day of filming; every film pays the same, so you want the **largest set of non-overlapping intervals**.

Heuristic 1 — EarliestJobFirst. *"It is best to work whenever work is available" — take the job with the earliest start date. Wrong: "accepting the earliest job might block us from taking many other jobs if that first job is long."* Skiena's counterexample is the epic **War and Peace**, which starts first and runs so long it kills every other prospect.

Heuristic 2 — ShortestJobFirst. The problem with *War and Peace* is that it's long, so take the shortest job each time. **Also wrong:** a single short job in the middle can block **two** others. *"While the maximum potential loss here seems smaller than with the previous heuristic, it can still limit us to half the optimal payoff."*

Exhaustive search works (2^n subsets — a million at $n = 20$, hopeless at $n = 100$), but is unnecessary:

*"Think about the first job to terminate — that is, the interval x whose right endpoint is left-most among all intervals. Other jobs may well have started before x , but all of these must at least partially overlap each other. Thus, we can select at most one from the group. The first of these jobs to terminate is x , so any of the overlapping jobs potentially block out other opportunities to the right of it. **Clearly we can never lose by picking x .**"*

That last sentence is Theorem 15.1 in one line.

CLRS Exercise 15.1-3 adds a third failed heuristic worth knowing: *always select the compatible activity that overlaps the fewest other remaining activities.* This one is much more plausible than the other two — and still wrong. The standard counterexample is a "ladder" shape where the minimum-conflict activity sits in the middle and splits the optimum.

HEURISTIC	CORRECT?	KILLER
earliest finish time	✓	Theorem 15.1
latest start time	✓ (Exercise 15.1-2)	symmetric proof
earliest start time	✗	one long early job
shortest duration	✗	one short middle job blocks two
fewest conflicts	✗	a crafted ladder

C++ Implementation

```
#include <algorithm>
#include <climits>
#include <numeric>
#include <vector>
```

```

struct Activity {
    int start, finish;
};

// GREEDY-ACTIVITY-SELECTOR: always take the compatible activity
// that finishes first.
vector<int> activitySelect(vector<Activity> a) {
    const int n = (int)a.size();
    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(),
        [&](int i, int j) { return a[i].finish < a[j].finish;
    });
    vector<int> chosen;
    int lastFinish = INT_MIN;
    for (int i : idx)
        if (a[i].start >= lastFinish) { chosen.push_back(i);
lastFinish = a[i].finish; }
    return chosen;
}

// Interval-graph colouring (Exercise 15.1-4): fewest lecture halls
// = max overlap.
int minLectureHalls(const vector<Activity>& a) {
    vector<pair<int, int>> events; // (time, +1 start
// -1 finish)
    for (const auto& x : a) { events.push_back({x.start, +1});
events.push_back({x.finish, -1}); }
    sort(events.begin(), events.end()); // finishes
// (-1) sort before starts at equal time
    int cur = 0, best = 0;
    for (const auto& e : events) { cur += e.second; best =
max(best, cur); }
    return best;
}

```

Verified: on CLRS's 11-activity instance it returns exactly $\{a_1, a_4, a_8, a_{11}\}$; across 400 random instances it matches exhaustive 2^n search every time, while earliest-start and shortest-job were strictly worse on many of them (on Skiena's *War and Peace* shape, earliest-start gets 1 where the optimum is 3). The lecture-hall greedy was verified to equal

the maximum number of simultaneously live activities on 300 instances — on the CLRS set it reports **6 halls**, a nice contrast with the **4** activities one room can host.

On minLectureHalls: the answer is exactly the **maximum overlap** at any instant. Sorting $(\text{time}, \pm 1)$ pairs makes finishes (-1) come before starts $(+1)$ at the same timestamp, which is correct for half-open intervals $[s, f)$ — a room freed at time t can be reused at time t . **That tie-break is the whole bug surface of this problem.**

Exercise 15.1-5 is the important negative result. Give each activity a value v_i and maximize total value rather than count. Greedy dies immediately — one high-value long activity can beat many cheap short ones. The fix is DP: sort by finish time, binary-search for the last compatible predecessor $p(i)$, and take $\text{best}[i] = \max(\text{best}[i-1], \text{best}[p(i)] + v_i)$ in $O(n \lg n)$. Weighted interval scheduling is the canonical "looks greedy, isn't" problem, and it is asked constantly in interviews.

Part 2 — Elements of the Greedy Strategy

The two design procedures

The long way (what §15.1 actually did — it exposes the DP underneath):

1. Determine the optimal substructure.
2. Develop a recursive solution.
3. Show that if you make the greedy choice, **only one subproblem remains**.
4. Prove that it is always **safe** to make the greedy choice. (3 and 4 in either order.)
5. Develop a recursive algorithm implementing the greedy strategy.
6. Convert it to an iterative algorithm.

The short way (what you actually do in practice):

1. **Cast the problem so that you make a choice and are left with exactly one subproblem.**
2. **Prove the greedy choice is always safe** — there is always an optimal solution making it.
3. **Demonstrate optimal substructure**: an optimal solution to the remaining subproblem, combined with the greedy choice, gives an optimal solution to the original.

Step 3 *"implicitly uses induction on the subproblems to prove that making the greedy choice at every step produces an optimal solution."*

The two ingredients

Greedy-choice property. *You can assemble a globally optimal solution by making locally optimal (greedy) choices — choosing what looks best in the current problem **without considering results from subproblems.***

The choice **may depend on choices already made**, but **may not depend on future choices or on subproblem solutions**. That is the entire dividing line from DP.

How the proof always goes: *"the proof examines a globally optimal solution to some subproblem. It then shows how to modify the solution to substitute the greedy choice for some other choice, resulting in one similar, but smaller, subproblem."*

Optimal substructure. Same as DP — but you get to **assume you arrived at the subproblem by having made the greedy choice**, which makes the argument much lighter. You need only argue that *optimal subproblem solution + greedy choice = optimal overall*.

And the practical payoff: *"By preprocessing the input or by using an appropriate data structure (often a priority queue), you often can make greedy choices quickly."* Sorting once (activity selection, Kruskal, fractional knapsack) or a heap (Huffman, Prim, Dijkstra) is the entire implementation of most greedy algorithms.

The three standard proof techniques

CLRS emphasizes the first; the other two are the standard vocabulary and worth naming.

TECHNIQUE	SHAPE	EXAMPLE
Exchange argument	Take any optimum $\mathcal{O}$. Show you can transform $\mathcal{O}$ into one containing the greedy choice without making it worse . Induct.	Theorem 15.1; Huffman Lemma 15.2; Theorem 15.5
"Greedy stays ahead"	Show that after k steps the greedy partial solution is at least as good as any other solution's first k steps, by some measure.	interval scheduling by finish time; the k -th activity greedy picks finishes no later than the k -th of any other schedule
Structural / bound-matching	Prove a lower bound on any solution's cost, then show greedy achieves it.	lecture halls: max overlap is a lower bound, and greedy uses exactly that many

Greedy vs. dynamic programming: the two knapsacks

0-1 knapsack. A thief takes a subset of n items into a knapsack of capacity w ; item i is worth v_i and weighs w_i . Each item is taken whole or not at all.

Fractional knapsack. Same, but the thief may take **fractions** of items. *"You can think of an item in the 0-1 knapsack problem as being like a gold ingot and an item in the fractional knapsack problem as more like gold dust."*

Both have optimal substructure. Only the fractional one has the greedy-choice property.

The greedy strategy: compute v_i/w_i (value per pound), take as much as possible of the densest item, then the next, and so on. $O(n \lg n)$ by sorting. (Exercise 15.2-6: $O(n)$ using median-of-medians selection — the same weighted-selection trick as M05.)

CLRS's counterexample (Figure 15.3), which you should be able to recite:

ITEM	WEIGHT	VALUE	DENSITY
1	10	\$60	\$6/lb
2	20	\$100	\$5/lb
3	30	\$120	\$4/lb

Capacity 50. **Greedy takes item 1** (highest density) and then can fit only item 2 $\rightarrow$ \$160. Or items 1+3 $\rightarrow$ \$180. **The optimum is items 2+3 = \$220**, leaving the densest item behind. Fractionally, taking item 1, item 2, and $\frac{2}{3}$ of item 3 gives **\$240**.

Why the difference? *"Taking item 1 doesn't work in the 0-1 problem, because the thief is unable to fill the knapsack to capacity, and the empty space lowers the effective value per pound of the load."* In the 0-1 problem, **you must compare the subproblem that includes the item against the subproblem that excludes it before you can choose** — and that's overlapping subproblems, i.e. DP.

```
#include <algorithm>
#include <numeric>
#include <vector>

double fractionalKnapsack(vector<int> w, vector<double> v, double
cap) {
    const int n = (int)w.size();
    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(),                // greedy:
best value per pound first
        [&](int i, int j) { return v[i] / w[i] > v[j] / w[j];
    });
    double total = 0;
    for (int i : idx) {
        if (cap <= 0) break;
        const double take = min((double)w[i], cap);
        total += v[i] * take / w[i];
        cap -= take;
    }
    return total;
}
```

Verified: on CLRS's instance the 0-1 DP gives **220**, density-greedy gives **160**, and the fractional greedy gives **240**. Across 400 random 0-1 instances, density-greedy was **strictly suboptimal on 59 of them (15%)** — and the fractional optimum was always an upper bound on the 0-1 optimum, as it must be (it's the LP relaxation).

Part 3 — Huffman Codes

The problem

"Huffman codes compress data well: savings of 20% to 90% are typical." You have a file of characters and a table of frequencies; design a **binary character code** minimizing the encoded length.

CLRS's running example — a 100 000-character file over $\{a, \dots, f\}$:

	A	B	C	D	E	F
Frequency (thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Fixed-length: $3 \times 100\,000 = 300\,000$ bits. Variable-length:

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \times 1000 = 224,000 \text{ bits}$$

A 25% saving, and it is optimal.

Prefix-free codes and the tree representation

A **prefix-free code** is one in which no codeword is a prefix of another. (*A prefix-free code can always achieve the optimal compression among any character code, so restricting to them loses nothing.*)

Why prefix-free matters: decoding becomes unambiguous. The codeword beginning an encoded file can only be one thing; strip it and repeat. `100011001101` parses uniquely as `100 · 0 · 1100 · 1101 = cafe`.

Represent the code as a **binary tree with the characters at the leaves**: `0` = go left, `1` = go right. (*These are **not** binary search trees — leaves aren't in sorted order and internal nodes hold no keys.*)

An optimal code is always represented by a full binary tree — every non-leaf has two children (Exercise 15.3-2). The fixed-length code above is *not* optimal precisely because its tree isn't full: it has codewords starting `10` but none starting `11`, i.e. a wasted branch. So the optimal tree has exactly $|C|$ leaves and $|C| - 1$ internal nodes.

The cost of a tree:

$$B(T) = \sum_{c \in C} c.\text{freq} \cdot d_T(c) \quad (15.4)$$

where $d_T(c)$ is the depth of c 's leaf — which is also **the length of c 's codeword**.

The algorithm

```
HUFFMAN(C)
1  n = |C|
2  Q = C                                // min-priority queue keyed
on freq
3  for i = 1 to n - 1
4      allocate a new node z
5      x = EXTRACT-MIN(Q)
6      y = EXTRACT-MIN(Q)
7      z.left = x
8      z.right = y
9      z.freq = x.freq + y.freq
10     INSERT(Q, z)
11 return EXTRACT-MIN(Q)                // the root is the only node
left
```

→ **C++ implementation:** [A2 HUFFMAN](#)

Bottom-up: start with $|C|$ leaves, perform $|C| - 1$ merges. **Each merge combines the two least frequent objects.** Skiena's one-paragraph version says exactly the same thing: *"Sort the symbols in increasing order by frequency. We merge the two least-frequently used symbols x and y into a new symbol xy , whose frequency is the sum of its two child symbols... We now repeat this operation $n - 1$ times until all symbols have been merged together."*

Running time. BUILD-MIN-HEAP is $O(n)$; the loop runs $n - 1$ times with $O(\lg n)$ heap operations $\Rightarrow O(n \lg n)$.

(The left/right assignment is arbitrary — swapping any node's children gives a different code of the same cost.)

Correctness

Lemma 15.2 (greedy-choice property). Let x and y be two characters of **lowest** frequency. Then there exists an optimal prefix-free code in which the codewords for x and y **have the same length and differ only in the last bit** — i.e. they are sibling leaves of maximum depth.

Proof (the exchange argument, done twice). Take any optimal tree T . Let a, b be **any two sibling leaves of maximum depth** in T , WLOG $a.freq \leq b.freq$ and $x.freq \leq y.freq$. Since x, y have the two lowest frequencies, $x.freq \leq a.freq$ and $y.freq \leq b.freq$.

Swap a and x to get T' ; then swap b and y to get T'' . Compute the first swap's effect:

$$\begin{aligned} B(T) - B(T') &= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_{T'}(a) - a.freq \cdot d_{T'}(x) \\ &= (a.freq - x.freq)(d_T(a) - d_T(x)) \\ &\geq 0 \end{aligned}$$

Both factors are nonnegative: $a.freq - x.freq \geq 0$ because **x is a minimum-frequency leaf**, and $d_T(a) - d_T(x) \geq 0$ because **a is a leaf of maximum depth**. So neither swap increases the cost. Since T was optimal, $B(T'') = B(T)$, and T'' is an optimal tree with x, y as sibling leaves of maximum depth. ■

That two-factor product is the whole proof. Memorize its shape: *swapping a light element toward a deep position and a heavy element toward a shallow one never costs more.*

Lemma 15.3 (optimal substructure). Let x, y be minimum-frequency characters. Let $C' = (C - \{x, y\}) \cup \{z\}$ with $z.freq = x.freq + y.freq$. If T' is an optimal tree for C' , then T — obtained by replacing z 's leaf with an internal node having children x and y — is optimal for C .

Proof. First, express $B(T)$ in terms of $B(T')$. For every $c \notin \{x, y\}$ the depth is unchanged. And since $d_T(x) = d_T(y) = d_{T'}(z) + 1$:

$$\begin{aligned} x.freq \cdot d_T(x) + y.freq \cdot d_T(y) &= (x.freq + y.freq)(d_{T'}(z) + 1) \\ &= z.freq \cdot d_{T'}(z) + (x.freq + y.freq) \end{aligned}$$

so $B(T) = B(T') + x.freq + y.freq$.

Now by contradiction: suppose some T'' has $B(T'') < B(T)$. By Lemma 15.2 we may assume T'' has x, y as siblings. Collapse them into a leaf z to get T''' ; then $B(T''') = B(T'') - x.freq - y.freq < B(T) - x.freq - y.freq = B(T')$, contradicting the optimality of T' . ■

Theorem 15.4. HUFFMAN produces an optimal prefix-free code. (Immediate from 15.2 and 15.3.) ■

Exercise 15.3-4 is the identity that makes the greedy view precise: $B(T)$ equals the sum, over all internal nodes, of the combined frequencies of that node's two children — i.e. the total cost of the tree is the sum of the costs of its mergers. So "merge the two cheapest" is greedy in the most literal sense: *of all possible mergers at each step, HUFFMAN chooses the one that incurs the least cost.*

C++ Implementation

```
#include <algorithm>
#include <functional>
#include <map>
#include <queue>
#include <string>
#include <vector>

struct HuffNode {
    long long freq;
    char ch = 0;
    int left = -1, right = -1;
};

struct HuffmanResult {
    long long cost; // total
    encoded bits
    map<char, string> code;
    vector<HuffNode> nodes;
    int root = -1;
};

HuffmanResult huffman(const map<char, long long>& freq) {
    HuffmanResult out{0, {}, {}, -1};
    if (freq.empty()) return out;

    using Item = pair<long long, int>; //
    (frequency, node index)
    priority_queue<Item, vector<Item>, greater<Item>> q;
    for (const auto& kv : freq) {
        out.nodes.push_back({kv.second, kv.first, -1, -1});
        q.push({kv.second, (int)out.nodes.size() - 1});
    }
}
```

```

    }
    if (q.size() == 1) { //
degenerate single-symbol alphabet
        out.root = q.top().second;
        out.code[out.nodes[out.root].ch] = "0";
        out.cost = out.nodes[out.root].freq;
        return out;
    }
    while (q.size() > 1) { // n-1
merges
        const auto x = q.top(); q.pop();
        const auto y = q.top(); q.pop();
        out.nodes.push_back({x.first + y.first, 0, x.second,
y.second});
        q.push({x.first + y.first, (int)out.nodes.size() - 1});
    }
    out.root = q.top().second;

    string path;
    function<void(int)> walk = [&](int u) {
        if (out.nodes[u].left < 0) { // leaf
            out.code[out.nodes[u].ch] = path;
            out.cost += out.nodes[u].freq * (long long)path.size();
            return;
        }
        path.push_back('0'); walk(out.nodes[u].left);
        path.pop_back();
        path.push_back('1'); walk(out.nodes[u].right);
        path.pop_back();
    };
    walk(out.root);
    return out;
}

string huffmanEncode(const HuffmanResult& h, const string& text) {
    string bits;
    for (char c : text) bits += h.code.at(c);
    return bits;
}

string huffmanDecode(const HuffmanResult& h, const string& bits) {
    string out;

```



```

    int u = h.root;
    if (h.nodes[u].left < 0) { //
single-symbol alphabet
        for (size_t i = 0; i < bits.size(); ++i) out +=
h.nodes[u].ch;
        return out;
    }
    for (char b : bits) {
        u = (b == '0') ? h.nodes[u].left : h.nodes[u].right;
        if (h.nodes[u].left < 0) { out += h.nodes[u].ch; u =
h.root; }
    }
    return out;
}

```

Implementation notes.

- Nodes live in a `std::vector` and are referenced by **index**, not pointer — no allocation churn, no ownership question, and the whole tree is one contiguous block.
- The **single-symbol alphabet** is a real edge case that crashes naive implementations: with one character there are zero merges, so the root is a leaf and the codeword would be the empty string. Assign it `"0"` explicitly.
- `std::priority_queue` with `std::greater` is a min-heap; the pair ordering breaks frequency ties by node index, which makes the output **deterministic** (important for testing, and for encoder/decoder agreement).

Verified: on CLRS's frequency table it produces cost **224** (thousand bits) against 300 for fixed-length, with codeword lengths exactly `a:1, b:3, c:3, d:3, e:4, f:4`; the code was checked to be genuinely prefix-free by comparing all pairs, and `encode ◦ decode` round-trips. Against a brute force over **all merge orders** (valid by Exercise 15.3-4), it matched on 200 random alphabets of up to 7 symbols. **Exercise 15.3-3** (Fibonacci frequencies `1,1,2,3,5,8,13,21`) produces codeword lengths `1,2,3,4,5,6,7,7` — the maximally unbalanced "bamboo" tree, because each Fibonacci number exceeds the sum of all smaller ones minus one, so every merge involves the running total.

Skiena's three practical caveats

Worth having, because they are the reason Huffman is *not* what your `.zip` file uses:

1. **Two passes over the document on encoding** — one to build the frequency table, one to encode.
2. **The coding table must be stored with the document**, "*which eats into any space savings on short documents.*" (Exercise 15.3-5: you can transmit the tree structure in $2^n - 1$ bits plus $n \lceil \lg n \rceil$ bits for the symbols.)
3. **Huffman only exploits non-uniform symbol distributions.** "*Adaptive algorithms can recognize the higher-order redundancies such as in 0101010101....*"

The alternative Skiena contrasts it with is **Lempel–Ziv** (and LZW), which "*compress text by building a coding table on the fly as we read the document... A clever protocol ensures that the encoder and decoder both work with the exact same code table, so no information is lost.*" His verdict: "*Adaptive algorithms usually prove to be the right answer for most problems.*"

Outside / Engineering Context

Real compressors use **both**: DEFLATE (gzip, zlib, PNG) runs LZ77 to find repeated substrings and then Huffman-codes the resulting literal/length/distance symbols. Modern codecs (zstd, brotli, and the entropy stage of most video codecs) replace Huffman with **arithmetic coding** or **asymmetric numeral systems (ANS)**, which can assign a symbol a fractional number of bits and therefore reach the true entropy $H = -\sum p_i \lg p_i$. Huffman's per-symbol integer-bit constraint costs it up to 1 bit per symbol — which is why **Exercise 15.3-7** holds: if all 256 byte values are roughly equally common (max frequency $< 2 \times$ min frequency), Huffman does no better than a flat 8-bit code.

Exercise 15.3-8 is the fundamental limit and worth knowing by heart: no lossless compression scheme can guarantee that every input file produces a shorter output. The counting argument: there are 2^n files of length n but only $2^n - 1$ files of length $< n$ in total, so no injection into shorter files exists. **Every compressor makes some inputs longer.**

Part 4 — Offline Caching and Furthest-in-Future

The problem

A cache holds k blocks and starts empty. A program requests blocks $b_1, \dots, b_n$. Each request is a **hit** (already cached), or a **miss**. On a miss with a full cache, some block must be **evicted**. Minimize the total number of misses.

A miss occurring while the cache is still filling up is a **compulsory miss** — unavoidable, since no prior decision could have helped.

Caching is normally an *online* problem — you don't know the future. Here we solve the **offline** version, where the whole request sequence is known. **Three reasons that's worth doing:**

1. Sometimes you really do know the sequence in advance — e.g. main memory as a cache over disk, with algorithms that plan all reads and writes ahead.
2. The offline optimum is the **baseline for measuring online algorithms** (competitive analysis, CLRS §27.3 — M24).
3. It models real problems: *"you know in advance a fixed schedule of n events at known locations... you are managing a group of k agents... you want to minimize the number of times that agents have to move."* Agents = blocks, events = requests, moving an agent = a miss.

The strategy and the proof

Furthest-in-future (Belady's rule): evict the cached block whose next access comes latest in the request sequence (or never). *"Intuitively, this strategy makes sense: if you're not going to need something for a while, why keep it around?"*

Optimal substructure. Define subproblem (C, i) = processing requests $b_i, \dots, b_n$ with cache configuration C . If s is optimal for (C, i) and C' is the cache after handling b_i , then s 's tail must be optimal for $(C', i+1)$ — otherwise splice in a better tail and beat s .

Let $R_{\{C, i\}}$ be the set of configurations reachable from C after request b_i :

- **hit:** $R_{\{C, i\}} = \{C\}$;
- **miss, cache not full:** $R_{\{C, i\}} = \{C \cup \{b_i\}\}$;
- **miss, cache full:** $R_{\{C, i\}} = \{(C - \{x\}) \cup \{b_i\} : x \in C\}$ — k choices.

$\int 0$	if
$i = n \text{ and } b_n \in C$	
$ 1$	if
$i = n \text{ and } b_n \notin C$	
$\text{miss}(C, i) = \begin{cases} \text{miss}(C, i+1) \end{cases}$	if
$i < n \text{ and } b_i \in C$	
$\lfloor 1 + \min\{ \text{miss}(C', i+1) : C' \in R_{\{C, i\}} \}$	if
$i < n \text{ and } b_i \notin C$	

That's a correct DP — and its state space is exponential (all $C(m, k)$ cache configurations). The greedy-choice property is what rescues it.

Theorem 15.5 (greedy-choice property). On a miss with a full cache, let $z = b_m$ be the cached block whose next access is furthest in the future (adding a dummy request b_{n+1} for any block never referenced again). Then **evicting z is part of some optimal solution.**

Proof sketch (the exchange argument, at its most elaborate). Let s be optimal and suppose it evicts some $x \neq z$. Build s' that evicts z instead, then mimics s . The core invariant: **for every j from $i+1$ to m , the two caches differ in at most one block — $C_{\{s, j\}} = D_j \cup \{z\}$ and $C_{\{s', j\}} = D_j \cup \{y\}$ for some $y \neq z$.** s' is defined case by case (hit/miss $\times$ whether $b_j = y$ $\times$ whether s evicts z or some $w \in D_j$) so that the invariant is maintained and **whenever s gets a hit, so does s' .**

At request $b_m = z$ the two caches become identical, and from then on s' copies s . The only way s could come out ahead is if it hits on b_m while s' misses — and CLRS shows by contradiction that in that case some **earlier** request in $b_{\{i+1\}}, \dots, b_{\{m-1\}}$ must have been a miss for s and a hit for s' , compensating. *(The contradiction: if no such request existed, the cache difference would never change, so $C_{\{s', j\}} = D_j \cup \{x\}$ throughout; but z is requested after x by definition, so x must be requested somewhere in $b_{\{i+1\}} \dots b_{\{m-1\}}$ — and there s' hits while s misses.)* Hence s' incurs no more misses than s . ■

Combined with optimal substructure, furthest-in-future is optimal.

C++ Implementation and the online comparison

```
#include <algorithm>
#include <cstdint>
#include <set>
#include <vector>
```

```

// furthest-in-future: evict the cached block whose next use is
latest (or never).
int furthestInFutureMisses(const vector<int>& req, int k) {
    set<int> cache;
    int misses = 0;
    for (size_t i = 0; i < req.size(); ++i) {
        if (cache.count(req[i])) continue;           // hit
        ++misses;
        if ((int)cache.size() < k) { cache.insert(req[i]);
continue; }
        int victim = -1;
        size_t victimNext = 0;
        for (int b : cache) {
            size_t next = req.size();               // "never
used again"
            for (size_t j = i + 1; j < req.size(); ++j)
                if (req[j] == b) { next = j; break; }
            if (victim < 0 || next > victimNext) { victim = b;
victimNext = next; }
        }
        cache.erase(victim);
        cache.insert(req[i]);
    }
    return misses;
}

int lruMisses(const vector<int>& req, int k) {
    vector<int> order;                             // front =
least recent
    int misses = 0;
    for (int b : req) {
        auto it = find(order.begin(), order.end(), b);
        if (it != order.end()) { order.erase(it);
order.push_back(b); continue; }
        ++misses;
        if ((int)order.size() == k) order.erase(order.begin());
        order.push_back(b);
    }
    return misses;
}

```

Verified: `furthestInFutureMisses` matched a memoized brute-force DP over all cache configurations on CLRS's own example sequence for $k = 1..4$, and on 300 random traces. On those same traces **LRU was strictly worse on 51/300 and FIFO on 56/300** — which is exactly Exercise 15.4-2's point.

*LRU is "furthest-in-past". It is the best implementable approximation to Belady's rule, and it is **not** optimal — but it is provably **k-competitive** (no online algorithm does better in the worst case; CLRS §27.3, M24). **Belady's rule is unimplementable online, and that is the whole point of studying it:** it defines the target that every real eviction policy is measured against. Every "hit rate vs. OPT" chart in a systems paper is measuring against this algorithm.*

Part 5 — The Greedy Catalog

Where greedy shows up in the rest of the course

ALGORITHM	GREEDY CHOICE	WHY IT'S SAFE	MODULE
Kruskal's MST	add the cheapest edge that doesn't create a cycle	the cut property / cycle property	M14
Prim's MST	add the cheapest edge leaving the current tree	the cut property	M14
Dijkstra's shortest paths	settle the unsettled vertex with the smallest tentative distance	nonnegative weights $\Rightarrow$ it can't improve later	M15
Huffman coding	merge the two least frequent symbols	Lemma 15.2	here
Fractional knapsack	take the densest item	exchange argument	here
Activity selection	take the earliest finish	Theorem 15.1	here
Offline caching	evict furthest-in-future	Theorem 15.5	here
Set cover approximation	take the set covering the most uncovered elements	gives an $H(n) \approx \ln n$ approximation, not optimal	M20
Coin change (canonical systems)	take the largest coin that fits	true for US coins & powers of c , false in general	here
Scheduling to minimize average completion time	shortest processing time first	exchange argument	here

Note the two entries where greedy is a *heuristic*, not an algorithm: set cover (an approximation with a proven ratio) and non-canonical coin systems (just wrong). Being precise about which kind you have is the professional skill.

Coin change: when greedy works and when it doesn't

```
#include <algorithm>
#include <vector>

int coinChangeGreedy(vector<int> coins, int n) {
    sort(coins.rbegin(), coins.rend());
    int used = 0;
    for (int c : coins) { used += n / c; n %= c; }
    return n == 0 ? used : -1;
}
```

```

int coinChangeDP(const vector<int>& coins, int n) {
    const int INF = 1000000;
    vector<int> best(n + 1, INF);
    best[0] = 0;
    for (int j = 1; j <= n; ++j)
        for (int c : coins)
            if (c <= j && best[j - c] + 1 < best[j]) best[j] =
best[j - c] + 1;
    return best[n] >= INF ? -1 : best[n];
}

```

- (a) Greedy is optimal for US coins $\{1, 5, 10, 25\}$.
- (b) Greedy is optimal for denominations $\{c^0, c^1, \dots, c^k\}$ for any $c > 1$ — because taking fewer than c of any denomination is forced (c of one equals one of the next), so the greedy count is the base- c representation.
- (c) Greedy **fails** in general. The minimal counterexample: $\{1, 3, 4\}$ and $n = 6$. Greedy takes $4 + 1 + 1 = 3$ coins; the optimum is $3 + 3 = 2$ coins.
- (d) The DP is $O(nk)$.

Verified: greedy == DP for US coins on all $n \leq 200$ and for powers of $c \in \{2, 3, 4, 5\}$ on all $n \leq 300$; and the $\{1, 3, 4\}$ counterexample reproduces exactly.

Coin systems where greedy is optimal are called *canonical*. Deciding whether a given system is canonical is itself nontrivial (there's an $O(k^3)$ test by Pearson). **Never assume your denominations are canonical.**

Other greedy exercises worth having in hand

- **Exercise 15.2-4 (water stops).** Skating across North Dakota, able to travel m miles per refill: **go as far as you can before refilling**. Exchange argument: any optimal solution's first stop can be pushed to the greedy stop without adding stops. $O(n)$.
- **Exercise 15.2-5 (unit intervals covering points).** Sort the points; place an interval starting at the leftmost uncovered point; repeat. The greedy interval covers a superset of what any other feasible interval covering that point could cover to the right.
- **Exercise 15.2-7 (rearrangement).** To maximize $\prod a_i^{b_i}$, sort both A and B and pair them in the same order — the **rearrangement inequality**, provable by exchange.

- **Problem 15-2 (average completion time). Shortest processing time first.**

Exchange argument: if a longer job precedes an adjacent shorter one, swapping them strictly reduces the total. $O(n \lg n)$ for the sort.

```
#include <algorithm>
#include <vector>

// Shortest-processing-time-first minimizes total (hence average)
// completion time.
long long totalCompletionTime(vector<long long> p) {
    sort(p.begin(), p.end());
    long long clock = 0, total = 0;
    for (long long x : p) { clock += x; total += clock; }
    return total;
}
```

Verified against all $n!$ permutations on 300 random instances.

Why SJF works, in one line: if job i runs in position k (1-indexed from the end), its processing time is counted k times in $\sum c_i$. So $\sum c_i = \sum_j (n - j + 1) \cdot p_{\{(j)\}}$, and to minimize a sum of products with a decreasing weight vector you pair the **smallest** p with the **largest** weight — rearrangement inequality again.

Outside / Engineering Context — matroids: the theory of when greedy works

CLRS 3e had a section on matroids that the 4th edition dropped. It answers the question this chapter otherwise leaves open — is there a general characterization of the problems greedy solves? — and it's worth 200 words.

A **matroid** is a pair $M = (S, \mathcal{I})$ where S is a finite set and $\mathcal{I}$ is a family of "independent" subsets satisfying:

- **hereditary:** if $B \in \mathcal{I}$ and $A \subseteq B$, then $A \in \mathcal{I}$;
- **exchange property:** if $A, B \in \mathcal{I}$ and $|A| < |B|$, then there is some $x \in B - A$ with $A \cup \{x\} \in \mathcal{I}$.

Theorem. For any weighted matroid, the greedy algorithm — sort elements by decreasing weight and add each one whose addition keeps the set independent — produces a **maximum-weight independent set**.

The canonical instance is the **graphic matroid**: $\mathcal{S}$ = the edges of a graph, $\mathcal{I}$ = the acyclic edge subsets (forests). Greedy on that **is exactly Kruskal's algorithm**, so MST optimality is a corollary of the matroid theorem rather than a one-off proof (M14). Task scheduling with deadlines and penalties is another matroid.

What this buys you in practice: if you can verify the two matroid axioms, greedy is optimal and you are done — no exchange argument needed. **What it does not buy you:** most greedy problems are not matroids (activity selection isn't; Huffman isn't; offline caching isn't). Matroids are a sufficient condition, not a characterization. The generalization that covers more ground is the **greedoid**, and for interval scheduling specifically the right lens is still the direct exchange argument.

Common Mistakes

MISTAKE	CONSEQUENCE
Presenting a greedy algorithm with no proof	It is a heuristic. Say so, or prove it.
"It works on my examples"	Skiena's whole point: earliest-start and shortest-job both work on many examples
Sorting by the wrong key	Activity selection by start or duration instead of finish — the single most common version of this bug
Assuming optimal substructure $\Rightarrow$ greedy works	0-1 knapsack has optimal substructure and greedy fails on it
Applying density-greedy to 0-1 knapsack	Suboptimal on ~15% of random instances (measured)
Assuming greedy coin change is optimal	Fails for $\{1, 3, 4\}$ at $n = 6$; only <i>canonical</i> systems work
Using greedy on weighted interval scheduling	Needs DP; this is a favorite interview trap
Getting the interval tie-break wrong	Half-open $[s, f)$ means a finish at time t frees the resource for a start at t ; sort -1 before $+1$
Forgetting the single-symbol alphabet in Huffman	Empty codeword; infinite loop or crash on decode
Believing LRU is optimal	It isn't — Belady's rule is (offline). LRU is k -competitive
Claiming a compressor shrinks every file	Impossible by counting (Exercise 15.3-8)
Proving "the greedy choice is in every optimal solution"	Too strong and usually false. You need " in some optimal solution. "

Recognition Patterns

Greedy is plausible when:

- there is a natural **ordering** (by finish time, weight, density, deadline, frequency) that makes a "best next" obvious;
- committing to that choice **provably never closes off** an optimum;
- after the choice, **exactly one** subproblem of the same shape remains;
- you can imagine the exchange argument before writing any code.

Greedy is probably wrong when:

- the objective is a **weighted** sum rather than a count (weighted interval scheduling, 0-1 knapsack);
- items are **indivisible** and the budget can be left partly unused;
- a choice **consumes a shared resource** that later choices also need (the longest-simple-path failure from M11);
- the problem asks you to **partition or assign** rather than **select in order**;
- your candidate ordering has more than one plausible competitor and you can't rule the others out.

The interview move: when you spot a greedy candidate, **immediately try to break it.**

Draw three intervals; try one long and two short; try one heavy and two light. If it survives sixty seconds of counterexample hunting (the M01 method: think small, go for a tie, seek extremes), sketch the exchange argument. If it dies, you now know the DP state, because *the reason greedy failed tells you what the state has to remember.*

Complexity Summary

PROBLEM	ALGORITHM	TIME	CORRECT?
Activity selection	earliest finish first	$\Theta(n)$ after sort, $O(n \lg n)$ total	✓ (Thm 15.1)
Weighted activity selection	DP with binary search	$O(n \lg n)$	greedy ✗, DP ✓
Minimum lecture halls	sweep for max overlap	$O(n \lg n)$	✓
Fractional knapsack	densest first	$O(n \lg n)$, $O(n)$ with selection	✓
0-1 knapsack	DP	$O(nW)$ pseudo-poly	greedy ✗
Huffman code	merge two least frequent	$O(n \lg n)$	✓ (Thm 15.4)
Offline caching	furthest-in-future	$O(nk)$ naive, $O(n \lg n)$ with next-use indices	✓ (Thm 15.5)
Coin change	largest coin first	$O(k \lg k)$	✓ only for canonical systems
Coin change	DP	$O(nk)$	✓ always
Min average completion time	shortest job first	$O(n \lg n)$	✓
Unit-interval point cover	leftmost uncovered point	$O(n \lg n)$	✓
MST	Kruskal / Prim	$O(E \lg V)$	✓ (M14)
Set cover	most-uncovered first	$O(n \cdot m)$	approximation only, ratio $H(n)$

One-Page Recall

- **Greedy = make the locally best choice, then solve the one subproblem that remains.** Top-down. The choice may depend on past choices, **never** on future ones or subproblem solutions.
- **Two ingredients: greedy-choice property + optimal substructure.** DP needs the second only; greedy needs both.
- **The design procedure:** (1) cast it so one choice leaves one subproblem; (2) prove the greedy choice is **safe**; (3) show greedy choice + optimal subproblem solution = optimal.

- **"Safe" means: there exists *some* optimal solution containing the greedy choice.** Not every optimal solution.
 - **The proof is an exchange argument:** take any optimum, swap the greedy choice in, show it stays feasible and no worse. Two other flavors: "greedy stays ahead" and matching a structural lower bound.
 - **Activity selection:** sort by **finish time**, take the first compatible one each time. $\Theta(n)$ after sorting. Earliest **start**, **shortest** duration, and **fewest conflicts** all fail.
 - **Knapsack:** greedy solves the **fractional** version (gold dust), fails on **0-1** (gold ingots) — 220 vs 160 vs 240 on the CLRS instance. The reason: leftover capacity dilutes the load's effective density.
 - **Huffman:** merge the two lowest-frequency nodes, $n-1$ times, with a min-heap $\Rightarrow O(n \lg n)$. Cost $B(T) = \sum \text{freq} \cdot \text{depth} = \text{the sum of all merge costs}$. Optimal trees are **full**. Lemma 15.2's key line: $B(T) - B(T') = (a.\text{freq} - x.\text{freq})(d_T(a) - d_T(x)) \geq 0$.
 - **Huffman's limits:** two passes, must ship the table, and it only exploits **first-order** symbol statistics. And no lossless scheme shortens every file.
 - **Offline caching:** evict **furthest-in-future** (Belady). Provably optimal; unimplementable online; the yardstick every real policy is measured against. LRU = "**furthest-in-past**" — not optimal, but k -competitive.
 - **Coin change:** greedy is optimal for canonical systems (US coins, powers of c) and **wrong** in general — $\{1, 3, 4\}$, $n = 6$.
 - **Minimize average completion time:** shortest job first, by rearrangement/exchange.
 - **The one-line test:** *can I state the exchange argument in two sentences?* If not, look for a counterexample; if you find none but still can't prove it, use DP.
 - **And CLRS's warning:** *beneath every greedy algorithm, there is almost always a more cumbersome dynamic-programming solution.* If greedy is failing you, that DP is where to go.
-

Practice — where to drill this module

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
Activity selection, verbatim	435 · Non-overlapping Intervals · 646 · Maximum Length of Pair Chain	sort by finish time and sweep — <code>A1</code> as a submission, twice
The same greedy, disguised	452 · Minimum Number of Arrows to Burst Balloons	"minimum arrows" is "maximum non-overlapping" with the words changed; spotting that is the skill
Interval bookkeeping	57 · Insert Interval	greedy merge; the edge cases are the problem
Huffman, exactly	1167 · Minimum Cost to Connect Sticks	repeatedly merge the two smallest — this is <code>HUFFMAN</code> , with the tree thrown away
Greedy with an exchange proof	45 · Jump Game II · 55 · Jump Game	the greedy is two lines and the proof is the interview
Greedy that fails	322 · Coin Change	take the largest coin first and watch it break on <code>{1, 3, 4}</code> , target 6 — then write the DP (M11)
Greedy + heap	[253? use instead] 621 · Task Scheduler (if available) · 1046 · Last Stone Weight	1046 is Huffman's loop with <code>max</code> instead of <code>min</code>
Greedy inside a graph algorithm	1584 · Min Cost to Connect All Points	Kruskal and Prim are both greedy with a cut-property proof (M14)

Beyond LeetCode. [CSES Problem Set](#) — *Sorting and Searching* is largely greedy. [Codeforces greedy tag](#) — the largest tag on the site, and the one where "I have a hunch" most often meets a counterexample.

The drill that matters here is not coding — it is the two-minute counterexample hunt from M01. Before writing a greedy, try to break it with: ties, one huge element, one tiny element, and an instance where the locally best choice consumes a resource two later choices needed. If you cannot break it in two minutes, *then* look for the exchange argument.

C++ Toolkit for This Module

Language material from Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., §1.6.4 (function objects) and ch. 6 (priority queues).

1. Sorting by the right key is usually the whole algorithm

Greedy algorithms are overwhelmingly "sort, then sweep". The comparator carries the algorithm:

```
struct Job { int start, finish; int id; };

void sortForActivitySelection(vector<Job>& a) {
    // Sort by FINISH time. Sorting by start time, by duration, or
    // by
    // "fewest conflicts" all give WRONG answers on the
    // counterexamples in
    // section 1 of this module -- the comparator IS the
    // algorithmic choice.
    sort(a.begin(), a.end(),
        [](const Job& x, const Job& y) { return x.finish <
        y.finish; });
}
```

The comparator must be a **strict weak ordering** (M05 toolkit §1): `cmp(a,a)` must be `false`. `return x.finish <= y.finish;` is undefined behaviour and really does crash `std::sort` on large inputs.

2. `priority_queue` is a MAX-heap; Huffman needs a MIN-heap

```
void heapDirections() {
    priority_queue<int> maxq; //
    top() = largest
    priority_queue<int, vector<int>, greater<int>> minq; //
    top() = smallest
    (void)maxq; (void)minq;
}
```


The comparator is the **third** template argument, so you must spell out the container (`vector<int>`) even though you did not want to change it. `HUFFMAN` extracts the two **smallest** frequencies, so it needs the second form — and getting this backwards produces a valid-looking tree with the *worst* possible cost.

3. Comparing pointers in a priority queue

Huffman's queue holds tree **nodes**, not numbers. A `priority_queue<DemoNode*>` would order by *pointer address* — arbitrary, and different on every run:

```
struct DemoNode { long long freq; char ch; DemoNode *left =  
    nullptr, *right = nullptr; };  
  
struct ByFreq {                                     // a function object [Weiss  
    1.6.4, p.42]  
    bool operator()(const DemoNode* a, const DemoNode* b) const {  
        return a->freq > b->freq;                // `>` because priority_queue  
    is a MAX-heap  
    }                                             // and we want the MINIMUM on  
    top  
};  
using DemoQueue = priority_queue<DemoNode*, vector<DemoNode*>,  
    ByFreq>;
```

Note the inversion: to get a min-heap out of a max-heap you supply a comparator that reports the *reverse* order. This is the single most confusing line in C++ heap code, and writing it out as a named functor rather than an inline lambda makes it legible.

4. Deterministic tie-breaking

When two nodes have equal frequency, the order is arbitrary — and different orders give different (equally optimal) trees. That is fine mathematically and a nuisance in tests. Add a tiebreak field if you need reproducible output:

```

struct ByFreqThenId {
    bool operator()(const pair<long long,int>& a, const pair<long
long,int>& b) const {
        if (a.first != b.first) return a.first > b.first;
        return a.second > b.second;    // stable, reproducible
across runs
    }
};

```

5. Owning a tree built inside a function

HUFFMAN allocates `n-1` internal nodes with `new`. Returning a raw `HuffNode*` makes ownership ambiguous — who calls `delete`? Two honest options:

```

// (a) an arena: all nodes live in one vector, freed together;
children are INDICES
struct ArenaNode { long long freq; int ch; int left = -1, right =
-1; };
// (b) unique_ptr, so the tree is destroyed automatically when the
root dies
struct OwnedNode { long long freq; int ch; unique_ptr<OwnedNode>
left, right; };

```

The appendix uses the **arena** form: it is the one that survives contact with a competitive-programming judge, has no allocation churn, and makes the whole tree trivially copyable and destroyable. It also sidesteps the Big-Five entirely (M06 toolkit §2).

6. `long long` for accumulated cost

$B(T) = \sum c.\text{freq} \cdot d_T(c)$ sums frequencies times depths. With realistic file frequencies this overflows 32 bits quickly. Same reflex as everywhere else in these notes.

Appendix — C++ for Every Pseudocode Block

A1 RECURSIVE-ACTIVITY-SELECTOR and GREEDY-ACTIVITY-SELECTOR

Pseudocode: §1, "Steps 3–4: the algorithms".

```
struct Activity {
    int start = 0, finish = 0;
    int id = 0;           // to report WHICH activities were
chosen
};

// RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)
// PRECONDITION: activities are sorted by finish time, and a
"virtual" activity
// a_0 with f[0] = 0 sits at index 0 so the first call k = 0 works
uniformly.
//
// The recursion is TAIL recursive -- the recursive call's result
is returned
//with only a prepend -- which is precisely why the iterative
version below
// exists and why CLRS presents both.
void recursiveActivitySelector(const vector<Activity>& a, int k,
int n,
                                vector<int>& chosen) {
    int m = k + 1;           // 1  m = k +
1
    while (m <= n && a[m].start < a[k].finish) // 2  find the
first activity in S_k
        m = m + 1;           // 3
    if (m <= n) {             // 4
        chosen.push_back(a[m].id); // 5  {a_m}
union RECURSIVE-...
        recursiveActivitySelector(a, m, n, chosen);
    }
    // 6  else return empty -- nothing left that starts after a_k
finishes
}

// GREEDY-ACTIVITY-SELECTOR(s, f, n): the same algorithm as a loop.
// Note that `m` never moves backwards across the whole run, so the
total work
// is Theta(n) -- the while loop of the recursive version and the
for loop here
```

```

// scan the SAME sequence exactly once. This is an amortized
// argument
// (M09): the inner scan looks nested but is bounded globally.
vector<int> greedyActivitySelector(vector<Activity> a) {
    if (a.empty()) return {};
    // Sort by FINISH time. This is the greedy choice, and it is
    // the only sort
    // key that works (toolkit 1).
    sort(a.begin(), a.end(),
        [](const Activity& x, const Activity& y) { return x.finish
        < y.finish; });

    vector<int> chosen;
    chosen.push_back(a[0].id);           // 1  A =
    {a_1}
    int k = 0;                          // 2  k = 1
    for (int m = 1; m < (int)a.size(); ++m) { // 3  for m =
    2 to n
        if (a[m].start >= a[k].finish) { // 4      is
    a_m compatible?
            chosen.push_back(a[m].id); // 5      A =
    A union {a_m}
            k = m;                      // 6      k =
    m
        }
    }
    return chosen;                       // 7  return A
}

```

Complexity. $\Theta(n)$ after sorting, $\Theta(n \lg n)$ including the sort. Space $\Theta(1)$ beyond the output.

Theorem 15.1 — why it is optimal. Let S_k be the activities that start after a_k finishes, and let a_m be the one in S_k with the **earliest finish time**. Then a_m is in *some* maximum-size subset of S_k .

Proof (exchange argument). Let A_k be any maximum-size compatible subset of S_k , and let a_j be its earliest-finishing member. If $a_j = a_m$, done. Otherwise replace a_j by a_m in A_k . The activities in $A_k - \{a_j\}$ all start at or after $f_j \geq f_m$, so they remain compatible with a_m . The new set has the same size and contains a_m . ■

The shape of that proof is the whole module: take an arbitrary optimal solution, *exchange* one element for the greedy choice, show the result is still feasible and no worse. If you can do that, the greedy is correct.

Why the other three rules fail (§1's table): *earliest start* lets one very long activity block everything; *shortest duration* lets a short activity wedged between two others kill both; *fewest conflicts* fails on a specific 11-activity instance. Only "earliest finish" is provably safe, because finishing soonest leaves the maximum possible room for everything after it.

A2 HUFFMAN

Pseudocode: §3, "The algorithm".

```
// ARENA representation (toolkit 5): every node lives in one
vector, children
// are INDICES rather than pointers. No new/delete, no Big-Five, no
leaks, and
// the whole tree is copyable and destructible for free.
struct HuffTree {
    struct Node {
        long long freq = 0;
        int ch = -1;           // -1 for an internal node;
    // else the character
        int left = -1, right = -1; // indices into `nodes`, -1 =
    none
    };
    vector<Node> nodes;
    int root = -1;
};

// HUFFMAN(C): C is a map from character to frequency.
HuffTree huffman(const vector<pair<char, long long>>& C) {
    HuffTree t;
    if (C.empty()) return t;

    // The priority queue holds (freq, nodeIndex). `greater<>`
    makes it a
    // MIN-heap (toolkit 2); the nodeIndex breaks ties
    deterministically
    // (toolkit 4), so the same input always yields the same tree.
    using Item = pair<long long, int>;
    priority_queue<Item, vector<Item>, greater<Item>> Q;
```

```

    for (const auto& [ch, f] : C) { // 2 Q = C
        t.nodes.push_back({f, (int)(unsigned char)ch, -1, -1});
        Q.push({f, (int)t.nodes.size() - 1});
    }

    // A single character is a special case the pseudocode glosses
    over: the
    // loop runs zero times and the "tree" is one leaf with no code
    at all.
    // Real encoders assign it the single bit 0.
    if (Q.size() == 1) { t.root = Q.top().second; return t; }

    for (size_t i = 1; i + 1 <= C.size() - 1 + 1 && Q.size() > 1;
    ++i) { // 3 n-1 times
        auto [fx, x] = Q.top(); Q.pop(); // 5 x =
        EXTRACT-MIN(Q)
        auto [fy, y] = Q.top(); Q.pop(); // 6 y =
        EXTRACT-MIN(Q)
        t.nodes.push_back({fx + fy, -1, x, y}); // 4,7,8,9
        z.left=x, z.right=y,
        // //
        z.freq = x.freq + y.freq
        Q.push({fx + fy, (int)t.nodes.size() - 1}); // 10
        INSERT(Q, z)
        // NOTE: push_back may REALLOCATE t.nodes, invalidating any
        pointer into
        // it (M09 toolkit 2). Storing INDICES rather than pointers
        is what makes
        // that harmless -- an index survives reallocation, a
        pointer does not.
    }
    t.root = Q.top().second; // 11 the last
    node is the root
    return t;
}

// Walk the tree to read off the codes: left = 0, right = 1.
void huffmanCodes(const HuffTree& t, int node, string prefix,
    map<char, string>& out) {
    if (node < 0) return;
    const auto& nd = t.nodes[node];

```

```

    if (nd.ch >= 0) { out[(char)nd.ch] = prefix.empty() ? "0" :
prefix; return; }
    huffmanCodes(t, nd.left, prefix + '0', out);
    huffmanCodes(t, nd.right, prefix + '1', out);
}

// B(T) = sum over characters of freq(c) * depth(c) -- the cost in
BITS.
long long huffmanCost(const HuffTree& t, int node, int depth = 0) {
    if (node < 0) return 0;
    const auto& nd = t.nodes[node];
    if (nd.ch >= 0) return nd.freq * depth;          // a leaf
contributes freq * depth
    return huffmanCost(t, nd.left, depth + 1) + huffmanCost(t,
nd.right, depth + 1);
}

// The same total, computed a completely different way: the sum of
the MERGE
// COSTS. Every merge of x and y adds (x.freq + y.freq), and each
character's
// frequency is counted once per merge it participates in -- which
is exactly
// its depth. B(T) = sum of the internal nodes' frequencies.
long long huffmanCostByMerges(const HuffTree& t) {
    long long total = 0;
    for (const auto& nd : t.nodes)
        if (nd.ch < 0) total += nd.freq;          // internal
nodes only
    return total;
}

```

Complexity. $O(n \lg n)$ — n inserts and $2(n-1)$ extract-mins, each $O(\lg n)$. With the frequencies already sorted it drops to $O(n)$ using two queues instead of a heap.

Why it is optimal — the two lemmas.

- **Lemma 15.2 (the greedy choice is safe).** Let x and y be the two characters of lowest frequency. There exists an optimal prefix code in which x and y have the same (maximum) depth and differ only in the last bit. *Proof:* take any optimal tree, let a and b be two sibling leaves of maximum depth, and **exchange** x with

a and y with b . The cost change is $(a.\text{freq} - x.\text{freq})(d_a - d_x) \geq 0$ in the right direction, so the new tree is no worse. ■

- **Lemma 15.3 (optimal substructure).** If T is optimal for the alphabet with x and y replaced by a merged character z of frequency $x.\text{freq} + y.\text{freq}$, then expanding z back into x and y gives an optimal tree for the original alphabet.

Theorem 15.4 is the two together: greedy choice + optimal substructure = HUFFMAN is optimal.

$B(T) = \sum \text{merge costs}$ is worth internalising — it is why "Minimum Cost to Connect Sticks" is Huffman, and why you can compute the answer without ever building the tree.

What Huffman does not do: it is optimal *among prefix-free codes that assign a whole number of bits per symbol*. Arithmetic coding beats it whenever the ideal code length is fractional (a symbol with probability 0.9 wants 0.15 bits and Huffman must give it 1). And Huffman assumes the frequencies are known in advance — adaptive Huffman and LZ-family coders exist for streams.

Next: *M13 — Graph Representation & Traversal (CLRS 20 + Skiena 7) — adjacency lists vs matrices, BFS and DFS with their full edge-classification machinery, topological sort, and strongly connected components.*